Análisis del Sistema POS y Arquitectura para Ingeniería de Software II

Universidad del Quindío

Facultad de Ingeniería - Programa de Ingeniería de Sistemas y Computación

Estudiante: Juan Esteban Herrera Cano

1. Artefactos a Entregar según la Guía

Para cumplir con los requerimientos de la actividad analítica del sistema POS implementado para Comercializadora EcoVida S.A.S., el presente reporte consolida los siguientes artefactos de ingeniería de software:

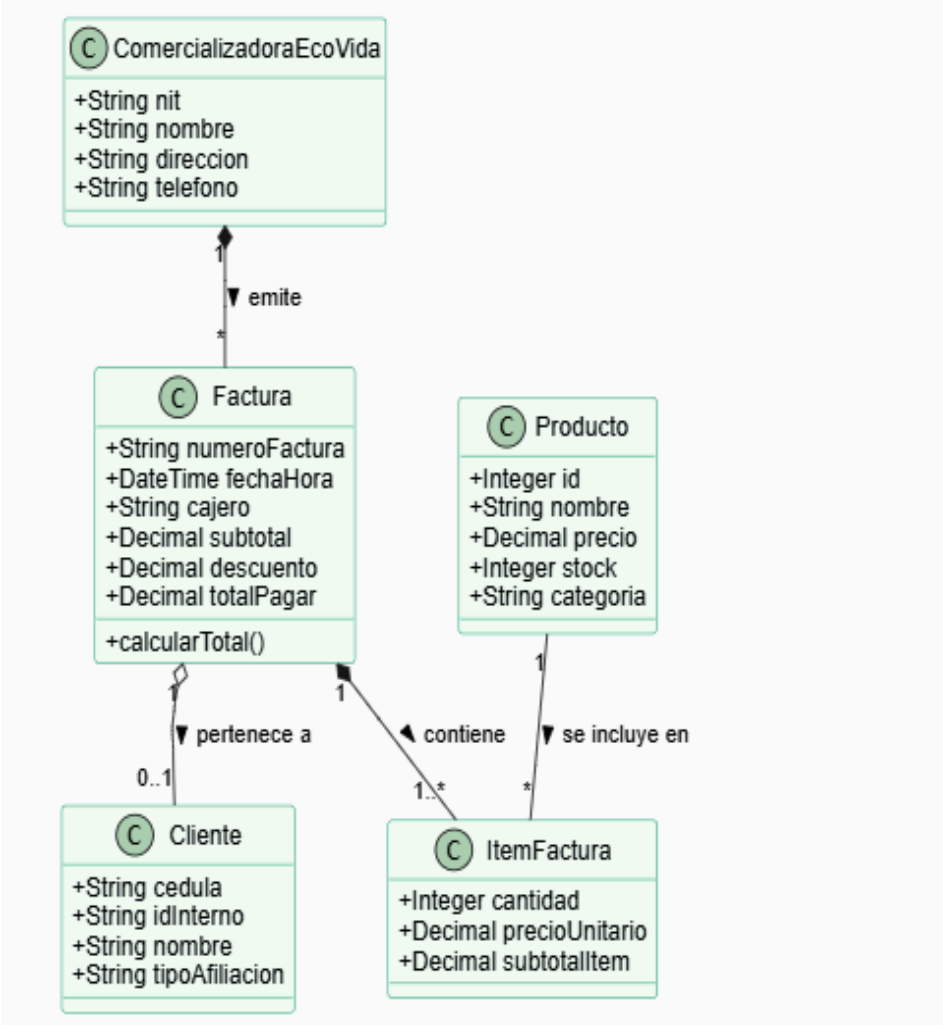
1. Lista de Entidades de Dominio y sus respectivos atributos tipados.
2. Diagrama de Clases UML (con relaciones, multiplicidades y operaciones principales).
3. Diagrama de Casos de Uso UML a nivel de sistema.
4. Lista de Requisitos Funcionales y No Funcionales con sus respectivos criterios de aceptación formalizados.
5. Arquitectura basada en el modelo $C4 + 1$: Diagrama C1 (Contexto), Diagrama C2 (Contenedores), Diagrama C3 (Componentes) y la vista de escenarios (+1).

2. Lista de Entidades de Dominio y Atributos

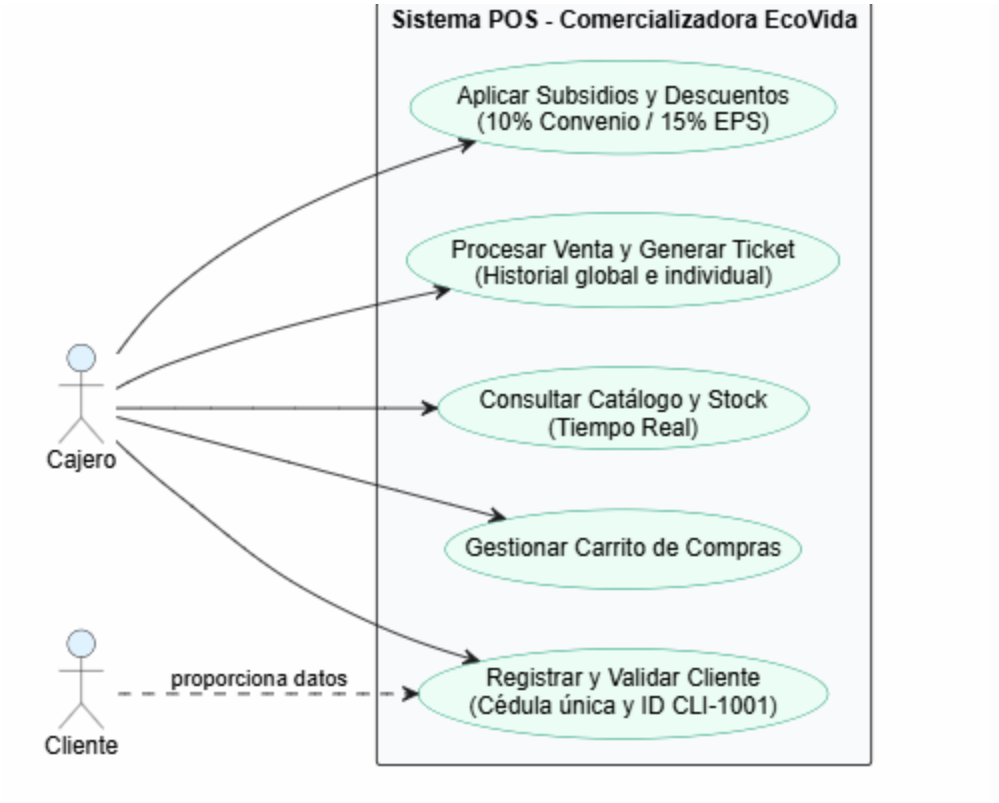
- **Comercializadora EcoVida**
  - nit (String)
  - nombre (String: "Comercializadora EcoVida S.A.S.")
  - direccion (String)
  - telefono (String)
- **Factura / Ticket**
  - numeroFactura (String)
  - fechaHora (DateTime)
  - cajero (String)
  - subtotal (Decimal)
  - descuento (Decimal)
  - totalPagar (Decimal)
- **ItemFactura (Detalle)**
  - cantidad (Integer)
  - precioUnitario (Decimal)
  - subtotalItem (Decimal)
- **Producto**

    - id (Integer)
    - nombre (String)
    - precio (Decimal)
    - stock (Integer)
    - categoria (String)
- **Cliente**
    - cedula (String, Única)
    - idInterno (String: Código consecutivo automático tipo CLI-1001)
    - nombre (String)
    - tipoAfiliacion (String: Particular, Convenio, EPS)

3. Diagrama de Clases



4. Diagrama de Casos de Uso



5. Requisitos Funcionales y No Funcionales

Requisitos Funcionales

- **RF01: Registro y Validación de Clientes**
  - *Descripción:* El sistema debe permitir registrar los compradores ingresando cédula, nombre y tipo de afiliación institucional.
  - *Criterio de Aceptación:* El backend valida que la cédula sea numérica y única, bloqueando duplicados e instituyendo automáticamente un código interno consecutivo (CLI-1001).
- **RF02: Control de Stock y Catálogo Sostenible**
  - *Descripción:* Visualización del catálogo de productos ecológicos con control estricto de inventario en tiempo real.
  - *Criterio de Aceptación:* Al agregar ítems al carrito, el sistema valida de forma automática que la cantidad solicitada no supere el stock disponible.
- **RF03: Cálculo Automatizado de Subsidios y Descuentos**
  - *Descripción:* El sistema debe aplicar de forma transparente reducciones porcentuales sobre el subtotal basadas en el perfil institucional del cliente.
  - *Criterio de Aceptación:* Aplica exactamente un 10% de descuento para clientes bajo modalidad "Convenio" y un 15% para modalidad "EPS".

- **RF04: Generación de Historial y Tiquetes**
    - *Descripción:* Registro persistente de cada factura o tiquete generado en la sesión comercial con trazabilidad global e individual.
    - *Criterio de Aceptación:* Permite consultar tanto el consolidado financiero global de ventas como el historial detallado de compras por cada cliente.

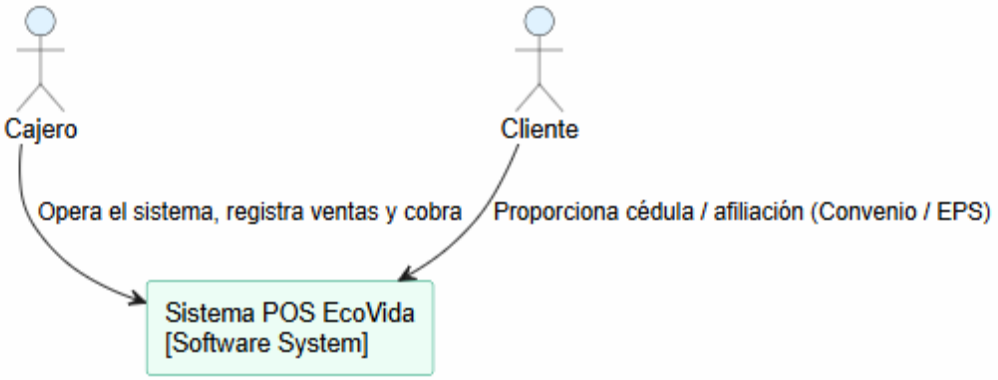
Requisitos No Funcionales

- **RNF01: Rendimiento y Fluidez Web**
    - *Descripción:* El tiempo de respuesta de la interfaz ante interacciones asíncronas no debe superar los 300 milisegundos.
    - *Criterio de Aceptación:* Con carritos de múltiples productos, la interfaz gráfica implementada con Tailwind CSS y FastAPI se mantiene fluida.
- **RNF02: Robustez y Manejo de Errores**
    - *Descripción:* Uso de notificaciones visuales en tiempo real y excepciones HTTP controladas para prevenir fallas de red silenciosas durante el proceso de caja.

6. Arquitectura del Sistema $(C4 + 1)$

Nivel C1: Contexto del Sistema

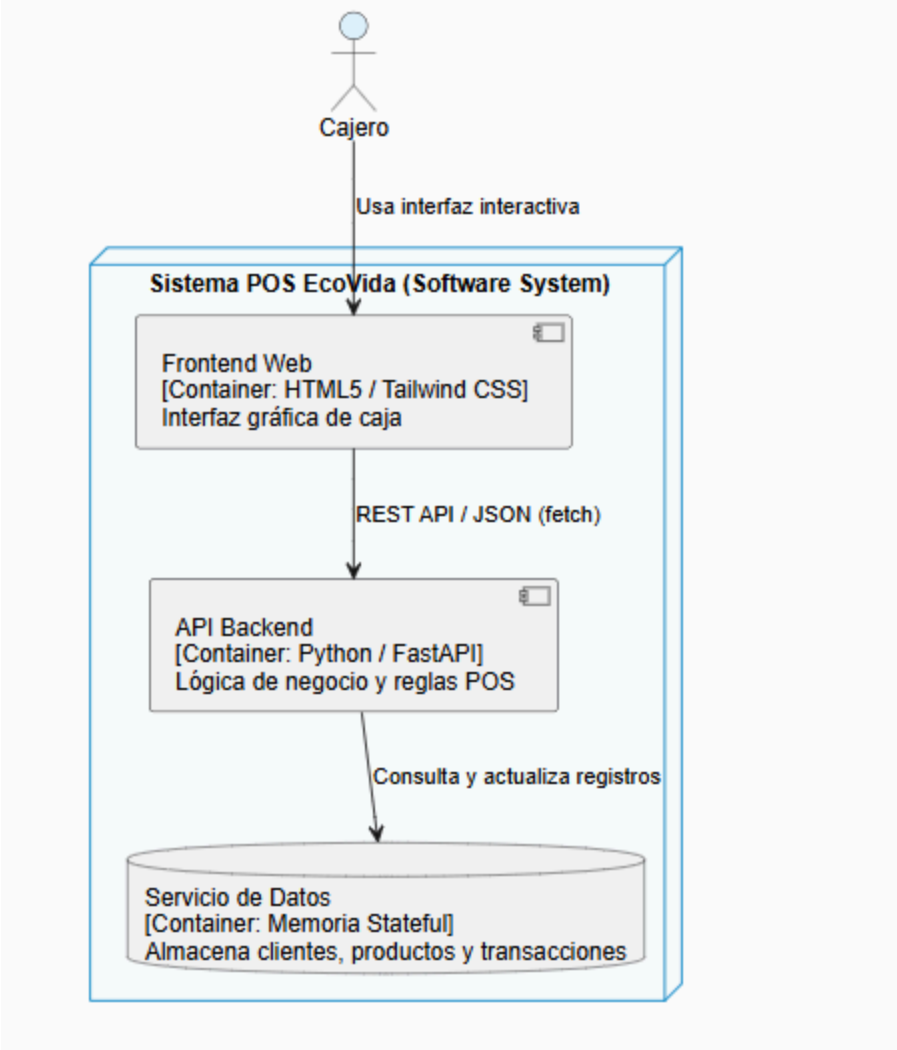
- **Cajero:** Opera la interfaz web del software POS para registrar las compras del proceso misional MI-02 y procesar cobros.
- **Cliente / Comprador:** Proporciona sus datos de identificación y se beneficia de los subsidios por EPS o Convenio.
- **Sistema POS EcoVida (Software Central):** Orquesta la lógica comercial, validaciones de unicidad, control de stock y cálculo de subsidios.



Nivel C2: Contenedores

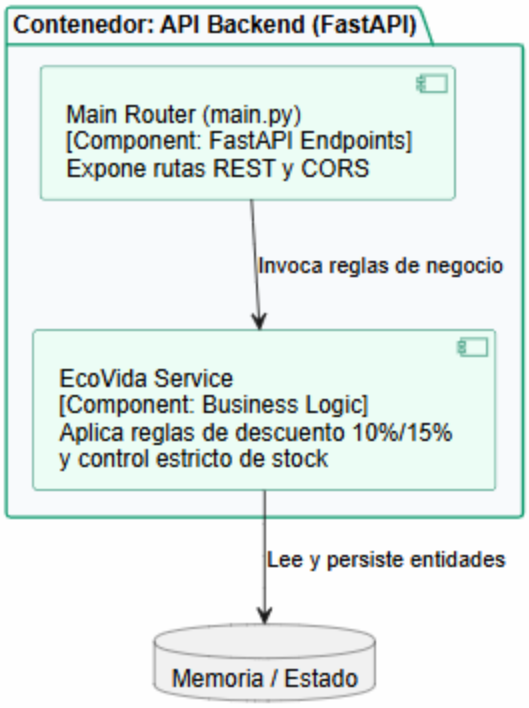
- **Frontend Web (Interfaz de Usuario):** Aplicación interactiva construida con HTML5, Tailwind CSS y JavaScript vanilla comunicada mediante REST API (fetch).

- **API Backend (Servidor FastAPI):** Capa desarrollada en Python encargada de ejecutar las reglas de negocio, validaciones Pydantic y endpoints HTTP.
- **Servicio de Datos en Memoria (EcoVidaService):** Estructura de estado stateful que almacena colecciones activas de clientes, productos y transacciones globales.



Nivel C3: Diagrama de Componentes (Dentro de la API Backend)

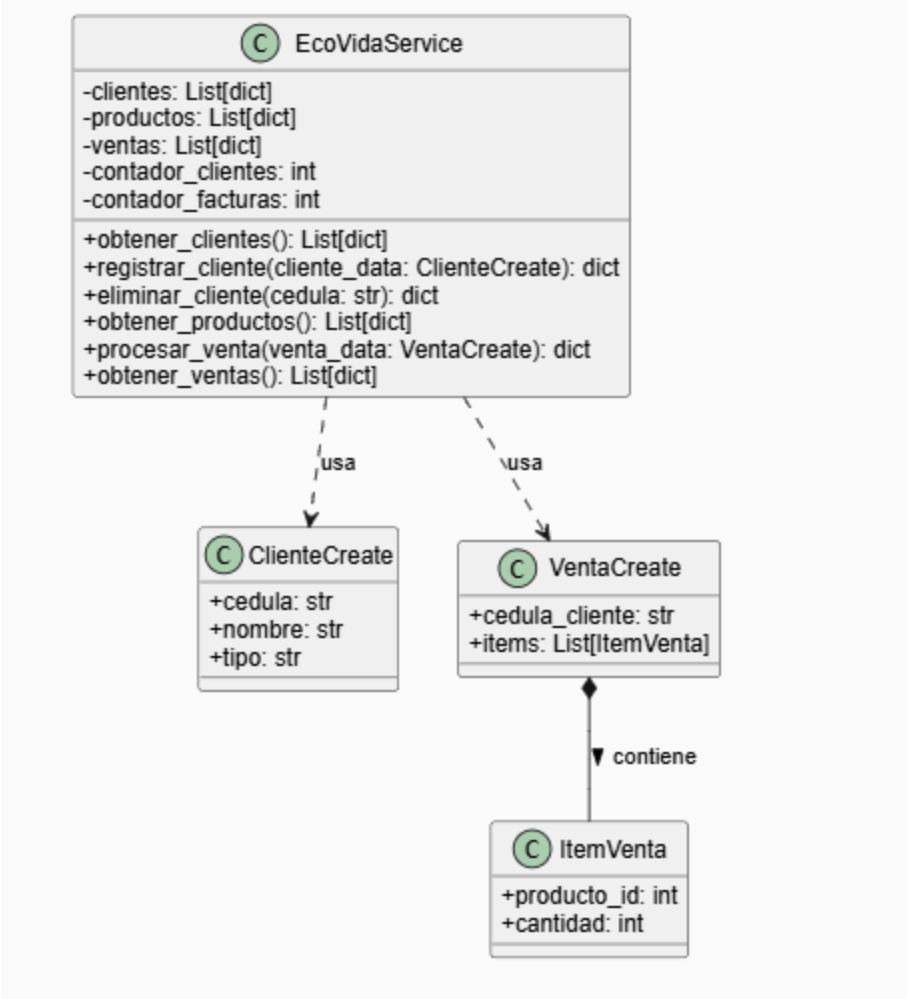
1. **Main Router (main.py):** Expone los endpoints REST y gestiona la pasarela de comunicación HTTP.
2. **Business Logic Service (EcoVidaService):** Contiene la lógica transaccional pura (registro con cédula única, control de stock, cálculo de descuentos 10%/15% y generación de historiales).



Nivel C4: Diagrama de Código (Estructura de Clases y Métodos)

Este nivel de abstracción desciende a la implementación interna y al detalle estático de los componentes del backend desarrollados en Python bajo el framework FastAPI. La arquitectura de código se compone de dos vertientes principales:

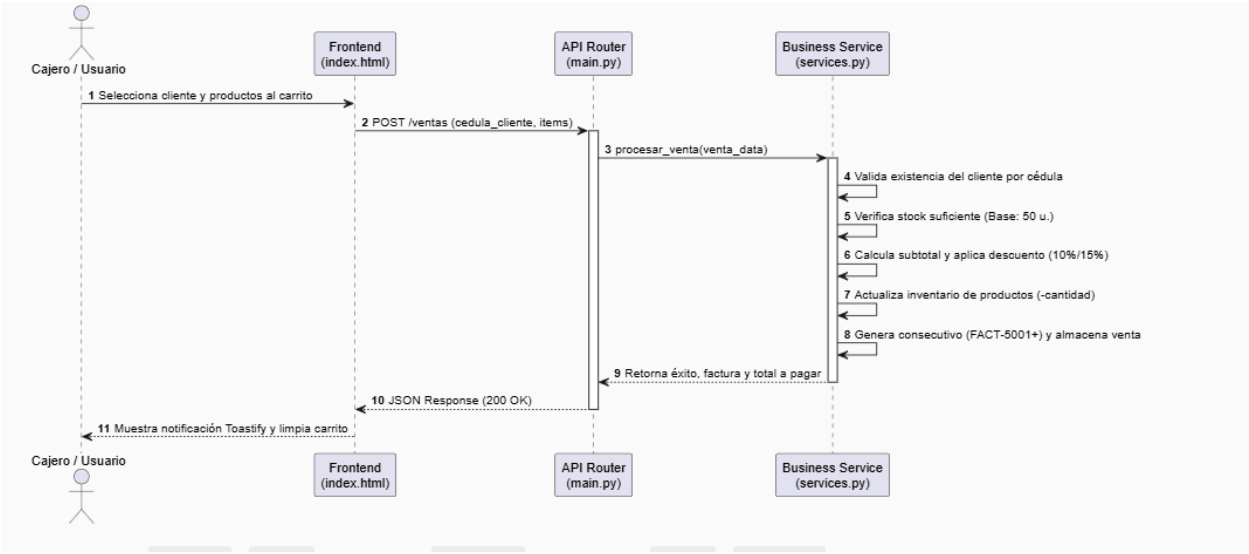
- **Esquemas de Transferencia y Validación (Pydantic):**
  - ClienteCreate: Estructura de validación encargada de asegurar la integridad de los datos en el registro de nuevos compradores (cedula, nombre y tipo de afiliación: *Particular*, *Convenio* o *EPS*).
  - ItemVenta y VentaCreate: Modelos anidados que estructuran la carga útil (*payload*) del carrito de compras, validando los identificadores de productos, las cantidades solicitadas y la cédula del cliente asociado.
- **Lógica Transaccional y de Servicio (EcoVidaService):**
  - Centraliza la persistencia en memoria y la ejecución de las reglas de negocio, incluyendo la validación de cédulas únicas, la gestión de un inventario unificado de 50 unidades base por referencia, el cálculo automatizado de descuentos institucionales y de salud (10% y 15%), y la emisión secuencial de facturación.



Vista "+1": Escenario Dinámico (Diagrama de Secuencia End-to-End)

Con el propósito de entrelazar la visión estática de la arquitectura (niveles C1 a C4) con el comportamiento operativo en tiempo de ejecución, esta vista implementa el modelo de casos de uso propuesto por Philippe Kruchten.

A través del siguiente diagrama de secuencia, se modela el flujo crítico y transaccional del sistema: desde que el usuario interactúa con la interfaz web para seleccionar los productos y el tipo de afiliación del cliente (ej. EPS con 15% de descuento), hasta que la solicitud es recepcionada por el enrutador HTTP de FastAPI (main.py), delegada al servicio de negocio para la validación de stock (baseline de 50 u.), cálculo de totales, actualización atómica del inventario y el retorno exitoso de la respuesta al cliente.


Cajero / Usuario
Frontend (index.html)
API Router (main.py)
Business Service (services.py)
1 Selecciona cliente y productos al carrito
2 POST /ventas (cedula_cliente, items)
3 procesar_venta(venta_data)
4 Valida existencia del cliente por cédula
5 Verifica stock suficiente (Base: 50 u.)
6 Calcula subtotal y aplica descuento (10%/15%)
7 Actualiza inventario de productos (-cantidad)
8 Genera consecutivo (FACT-5001+) y almacena venta
9 Retorna éxito, factura y total a pagar
10 JSON Response (200 OK)
11 Muestra notificación Toastify y limpia carrito
Cajero / Usuario
Frontend (index.html)
API Router (main.py)
Business Service (services.py)